

# RPA skalieren ohne Kontrollverlust

Governance, Audit Trail, Exceptions, Security by Design –  
und Change Management als Verantwortungsarchitektur.

*Lehrskript – Tag 12 von 16*

Lernpfad:  
Wissen → Anwendung → Management

Bearbeitungsdauer:  
1 Kurstag

Dozent:  
Can Siebert

Format:  
Theorie · Fallstudie · Coaching

---

## Lernziele dieses Tages

Tag 11 hat gezeigt, dass RPA als Steuerungshebel verstanden werden muss. Heute geht es darum, ein wachsendes RPA-Portfolio *auditfähig und sicher zu skalieren*: Governance leicht, aber verbindlich, Audit Trail als Standard, Exception Handling als Disziplin, Security by Design als Pflicht und Change Management als Teil des Designs – nicht als Nachtrag.

### L1 – Wissen (Reproduktion und Einordnung)

- Bot-Lifecycle und Governance-Bausteine kennen: Idea → Assessment → Design → Build → Test → Deploy → Run → Improve → Retire.
- Rollen und Rollenträger im RPA-Operating-Modell unterscheiden (Process Owner, Bot Owner, Developer, Reviewer, Security, Operations, Change).
- Audit-Trail-Anforderungen verstehen: Traceability, Aufbewahrung, Access-Kontrolle, Beweisführung.
- Exception-Klassen (Business, System, Data, Security) und Security-by-Design-Prinzipien (Least Privilege, SoD, Vault) einordnen.

### L2 – Anwendung (Transfer auf konkrete Situationen)

- Ein “Minimum Viable Governance”-Modell entwerfen (Rollen, Lifecycle, Freigaben, Standards).
- Audit-Trail-Standard inkl. Aufbewahrung und Access-Regeln festlegen.
- Exception-Codes und 5 Exception-Playbooks (Eskalation, Owner, SLA) ableiten.
- Security-Guardrails (Account-/Credential-Konzept, SoD, Logging-Integrität) anwenden.

### L3 – Management (Entscheidung und Verantwortung)

- Risk Appetite definieren und Bots in Low/Medium/High Risk klassifizieren.
- CoE vs. Föderation als bewusste Entscheidung treffen, mit Frühwarnsignalen.
- Fast-Track-Grenzen und Stop-Conditions verankern.
- Change Management von Beginn an im Operating Model verankern (Akzeptanz, Rollenwandel, Kommunikation).

#### Roter Faden

RPA skaliert dort, wo Governance *leicht* und Disziplin *konsequent* ist. Hundert Bots ohne Audit Trail sind ein Compliance-Albtraum – hundert Bots mit überreguliertem Prozess sind ein Innovationsfriedhof. Senior-Verantwortung heißt: das Gleichgewicht aus Geschwindigkeit und Nachweisbarkeit explizit zu entscheiden, nicht zu hoffen, dass es entsteht. (vgl. ISACA, 2018; Smeets et al., 2019)

## 1. Einstieg: Warum Skalierung neue Probleme erzeugt

Wer in einem Pilot fünf Bots betreibt, kennt jeden mit Namen. Wer in der Skalierung 80 Bots betreibt, kennt nur noch die, die ärgerlich sind. Diese qualitative Veränderung ist der eigentliche Skalierungssprung – nicht die Anzahl. Mit jeder Vervielfachung der Bots wachsen drei Klassen von Risiken nicht linear, sondern überproportional: Governance, Audit und Security.

### 1.1 Drei typische Skalierungs-Pathologien

1. **Wildwuchs.** Jeder baut, wie er will. Naming-Konventionen variieren, Logging ist unterschiedlich, Exception-Codes existieren in zehn Varianten. Eine zentrale Auswertung wird unmöglich.
2. **Compliance-Schock.** Der Auditor entdeckt im Audit-Sampling Bots ohne Logging, ohne Versionierung, mit shared Credentials. Folge: Stoppanordnung, Sonderprüfung, Reputationsschaden.
3. **Operationaler Stillstand.** Ein Portal-Update legt 15 Bots gleichzeitig lahm, niemand weiß, welche prioritär sind, alle Owner werden gleichzeitig kontaktiert. Reibung, Eskalationen, Vertrauensverlust.

### 1.2 Was Governance leisten muss

Governance ist nicht “Bürokratie für Auditoren”, sondern eine Sammlung von Mechanismen, die Skalierung möglich machen: (ISACA, 2018)

- **Vergleichbarkeit.** Bots werden so gebaut, dass sie sich vergleichen lassen (Naming, Logging, Versionierung).
- **Auffindbarkeit.** Wer welchen Bot wann gebaut hat, ist im zentralen Inventar nachvollziehbar.
- **Steuerbarkeit.** Bots können kontrolliert pausiert, neu deployed, abgeschaltet werden – ohne dass es von der Erinnerung einzelner Personen abhängt.
- **Lernfähigkeit.** Aus Incidents und Findings entstehen Verbesserungen des Standards – nicht nur Reparaturen einzelner Bots.

#### Eselsbrücke: V-A-S-L

Was eine Governance für skalierendes RPA leisten muss:

Vergleichbarkeit (einheitliche Standards)

Auffindbarkeit (zentrales Inventar)

Steuerbarkeit (kontrollierte Lifecycle-Operationen)

Lernfähigkeit (Standards entwickeln sich weiter)

*Wer V-A-S-L verfehlt, hat keine Governance – nur Dokumente.*

## 2. Bot-Lifecycle: vom Vorschlag bis zum Retirement

Jeder Bot durchläuft idealerweise einen klar definierten Lifecycle. Wer den Lifecycle überspringt oder verkürzt, holt sich später Probleme in Geschwindigkeit, Qualität oder Compliance. (Smeets et al., 2019, Kap. 4)

## 2.1 Die neun Phasen

1. **Idea.** Eine Idee aus dem Fachbereich. Noch ungeprüft.
2. **Assessment.** Eignung (Regelbasiert? Stabil? Volumen? Ausnahmen?) und Risikoklasse (Low/Medium/High) werden bestimmt.
3. **Design.** Anforderungen, Bot-Flow, Exception-Strategie, Logging-Konzept, Security-Anforderungen werden festgelegt.
4. **Build.** Entwicklung im Studio, gegen Standards.
5. **Test.** Unit, Integration, UAT (User Acceptance Test) mit den Fachbereichen.
6. **Deploy.** Übergabe in die produktive Orchestrierung, mit Smoke Tests.
7. **Run.** Laufender Betrieb mit Monitoring, Exception Handling, regelmäßigem Health-Check.
8. **Improve.** Verbesserungen aufgrund von Incidents, Exceptions, neuer Anforderungen.
9. **Retire.** Geplante Abschaltung mit Daten-Archivierung, Übergangs-Plan, Dokumentation.

## 2.2 Lebenszyklus-Disziplin in der Praxis

In gewachsenen Programmen sind oft die ersten und letzten Phasen schwach: Idea wird ohne Assessment direkt gebaut (“ich kenne den Prozess”), und Retire wird vergessen, sodass alte Bots Zombies werden – niemand betreibt sie aktiv, aber sie laufen und konsumieren Credentials und Logs.

Drei Disziplinen:

- **Intake-Pflicht.** Keine Entwicklung ohne Assessment-Eintrag. Auch wenn der Use Case “offensichtlich” wirkt – Assessment kostet 15 Minuten, spart später Tage.
- **Run-Hygiene.** Mindestens quartalsweise Review pro Bot: läuft er noch? wird er noch genutzt? sind die SLAs erfüllt? hat sich das Zielsystem geändert?
- **Retire-Konsequenz.** Ein Bot, der nicht mehr gebraucht wird, wird abgeschaltet. Nicht “vielleicht später nochmal” – mit Dokumentation und Daten-Archivierung.

## 3. Rollen und RACI in einem RPA-Operating-Modell

Ein wachsendes Programm braucht klare Rollen. Sechs Kernrollen reichen für die meisten Organisationen: (PMI, 2021)

### 3.1 Die sechs Kernrollen

- **Process Owner.** Fachlicher Eigentümer des Prozesses, in dem der Bot arbeitet. Definiert Anforderungen, akzeptiert Outputs, verantwortet fachliche Korrektheit.
- **Bot Owner.** Verantwortet einen konkreten Bot in Build und Run. Eine Person, nicht ein Verteiler.
- **Developer.** Baut den Bot im Studio, gegen Standards.
- **Reviewer / QA.** Prüft Bot gegen Naming, Tests, Logging, Security-Baselines vor Deployment.

- **Security Owner.** Definiert Baselines (Credentials, Vault, Logging, SoD) und prüft Ausnahmen.
- **Operations / Run.** Betreibt die Plattform (Orchestrator, Server, Updates), reagiert auf Incidents, verantwortet MTTR.

### 3.2 RACI für eine Bot-Änderung

|                    | P. Owner | Bot Owner | Dev | QA  | Sec | Ops |
|--------------------|----------|-----------|-----|-----|-----|-----|
| Anforderung        | A        | R         | C   | I   | I   | I   |
| Design (Low Risk)  | C        | R/A       | C   | I   | I   | I   |
| Design (High Risk) | C        | R         | C   | A   | A   | I   |
| Build              | I        | A         | R   | I   | I   | I   |
| Review / QA        | I        | C         | C   | R/A | C   | I   |
| Deploy             | I        | A         | C   | I   | I   | R   |
| Run / Monitoring   | I        | A         | I   | I   | I   | R   |
| Retire             | A        | R         | C   | I   | C   | C   |

Die Matrix zeigt: Low-Risk-Bots brauchen wenig Approval, High-Risk-Bots brauchen mehr – das ist *Risk-based Releases* in der RACI-Praxis.

### 3.3 Fast Track für kleine Bots

Nicht jeder Bot braucht das volle Programm. Wer das nicht differenziert, erstickt an seinem eigenen Prozess. Ein **Fast Track** für Low-Risk-Bots ist legitim, wenn:

- Das Risiko klar als Low klassifiziert ist (keine sensiblen Daten, keine finanziellen Buchungen, kein Compliance-Bezug).
- Mindestens eine QA-Prüfung erfolgt (Tests grün, Logging vorhanden).
- Der Bot Owner verbindlich benannt ist.
- Im Incident-Fall klare Eskalation existiert.

#### Fast Track ≠ no Track

Ein Fast Track ist keine Erlaubnis, alles wegzulassen – sondern eine Reduktion der Reviews. Das Logging-Minimum, der Bot Owner und die Tests sind nicht verhandelbar. Wer Fast Track als “schnell und schmutzig” liest, baut sich seine eigenen Audit-Findings.

## 4. Audit Trail Management: Beweisführung im Sekundentakt

Ein Bot, der hundert Buchungen pro Stunde macht, erzeugt potenziell hundert prüfungsrelevante Ereignisse pro Stunde. Wenn diese nicht systematisch dokumentiert sind, wird jede Audit-Anfrage zur Suchaktion. (ISACA, 2018)

### 4.1 Was ein Audit Trail beantworten muss

Der *klassische W-Fragen-Katalog* der Audit-Disziplin:

- **Wer?** Welcher Bot in welcher Version? Welcher menschliche Beteiligte bei manueller Übernahme?

- **Was?** Welche Aktion wurde ausgeführt? Welche Daten gelesen, welche geschrieben?
- **Wann?** Mit präzisiertem Zeitstempel inklusive Zeitzone.
- **Womit?** Welche Quelle (Mail-ID, Portal-Datei, Eingangs-Queue)?
- **Ergebnis?** Erfolgreich, in Queue, Exception (welcher Code)?
- **Begründung?** Bei Ausnahmen und manuellen Eingriffen: Warum?

## 4.2 Traceability über Systemgrenzen

Ein Bot bewegt sich häufig durch mehrere Systeme. Eine **Korrelations-ID** (Case-ID, Vorgangs-ID) verbindet alle Spuren des gleichen Vorgangs. Ohne sie bleiben Logs Inseln, mit ihr entsteht eine durchgängige Spur.

Drei Schichten:

- **Bot-Log.** Was hat der Bot intern getan?
- **Zielsystem-Log.** Welche Aktionen wurden in ERP/CRM/Portal ausgeführt? Idealerweise mit derselben Korrelations-ID.
- **Orchestrator-Log.** Welche Bots, welche Maschinen, welche Queues – übergreifend.

## 4.3 Aufbewahrung und Access

- **Aufbewahrungsdauer.** Je nach Regulatorik – typischerweise 2 bis 10 Jahre. Steuerrelevante Buchungen oft 10 Jahre, allgemeine IT-Logs eher 6–24 Monate.
- **Speicherort.** Zentral, mit Backup, mit Manipulationsschutz (idealerweise append-only oder mit kryptographischer Signatur).
- **Access-Kontrolle.** Wer darf Logs einsehen? Role-based: Bot Owner sieht “seine” Logs, Auditor sieht alle aggregiert, Security sieht relevante Security-Events.
- **Exportfähigkeit.** Für Audits muss eine kontrollierte Extraktion möglich sein – maschinenlesbar, mit definiertem Schema.

### Append-only als Schutzprinzip

Ein Audit Trail, der nachträglich geändert werden kann, ist keiner. Das Prinzip *Append-only* (Einträge können hinzugefügt, aber nicht geändert oder gelöscht werden) ist der einzige verlässliche Schutz gegen Manipulation – ob versehentlich oder vorsätzlich.

## 5. Exception Handling: kontrolliert scheitern

Ein Bot, der nicht weiß, was er bei Fehlern tun soll, ist ein Bot, der gefährlich ist. Eine systematische Exception-Strategie ist die Grundlage skalierender RPA. (Smeets et al., 2019, Kap. 6)

### 5.1 Drei Exception-Klassen

- **Business Exception.** Fachlich begründete Ausnahme: “ApprovalRequired”, “DuplicateInvoice”, “ContractExpired”. Erfordert fachliche Bearbeitung.

- **System Exception.** Technische Ausnahme: “ERPDowN”, “Timeout”, “ConnectionLost”. Erfordert Retry-Strategie oder Eskalation an IT.
- **Data Exception.** Datenqualitätsausnahme: “MissingCostCenter”, “InvalidIBAN”, “UnknownVendor”. Erfordert Datenkorrektur an der Quelle oder im Bot.

## 5.2 Das “Fail-safe”-Prinzip

Ein guter Bot scheitert *kontrolliert*: er bricht ab, schreibt einen Eintrag in die Exception Queue, dokumentiert den Code, übergibt an die zuständige Rolle – und lässt nicht halbfertige Zustände im Zielsystem zurück.

Drei Regeln:

1. **Transactional.** Wenn der Bot eine “Anlage” und eine “Freigabe” macht: entweder beide oder keine. Halbe Vorgänge sind die teuersten Bugs.
2. **Logged.** Jede Exception erzeugt einen Eintrag mit Code, Kontext und Bot-Version. Ohne Log ist sie nicht analysierbar.
3. **Routed.** Jede Exception-Klasse hat einen vorab definierten Owner und SLA. Niemand wartet, weil “vielleicht IT” zuständig sei.

## 5.3 Exception-Playbooks

Ein **Playbook** ist eine kurze Handlungsanweisung für eine Exception-Klasse:

- **Trigger.** Wann wird das Playbook aufgerufen?
- **Diagnose.** Welche Information benötigt der Bearbeiter sofort?
- **Aktionen.** Erste, zweite, dritte Handlungsoption.
- **Eskalation.** Wann an wen, mit welchen Informationen?
- **Schließung.** Wie wird der Vorgang final dokumentiert und in den Bot-Lauf zurückgegeben?

Beispielhaftes Playbook für “ERPDowN”:

1. Retry 3 Mal mit exponentiellem Backoff (1 min, 5 min, 15 min).
2. Wenn weiterhin Fehler: Ticket an IT mit Bot-ID, Vorgangsanzahl in Queue, Zeitstempel.
3. Eskalation nach 2 Stunden ohne IT-Response an IT-Lead.
4. Bei Wiederverfügbarkeit: Queue automatisch abarbeiten, Statusmeldung an Bot Owner.

## 5.4 Exception-KPIs

- **Exception Rate.** Anteil Vorgänge mit Exception. Zielmarke häufig  $\leq 10\text{--}12\%$  nach Stabilisierung.
- **MTTR pro Exception.** Wie lange dauert die Bearbeitung im Schnitt? Niedrige MTTR bedeutet eingespielte Playbooks.
- **Recurrence Rate.** Anteil wiederkehrender Exceptions. Hohe Quote = strukturelles Problem, niedrige Quote = Einzelfälle.

# 6. Security by Design: der Bot als privilegierter Mitarbeiter



Ein RPA-Bot ist nicht “nur eine Software” – er hat Zugriff auf Systeme, kann Daten lesen, kann Transaktionen auslösen. Aus Sicherheitssicht ist er ein *privilegierter Mitarbeiter*. Was für menschliche Privilegien gilt, gilt auch für Bots. (Saltzer & Schroeder, 1975; ISO/IEC, 2022)

### 6.1 Die Grundprinzipien (Saltzer & Schroeder, 1975)

Die 1975 von Saltzer und Schroeder formulierten Prinzipien sind heute immer noch das Fundament: (Saltzer & Schroeder, 1975)

- **Least Privilege.** Der Bot hat nur die Rechte, die er für seine Aufgabe braucht – nicht mehr.
- **Economy of Mechanism.** Das Sicherheitsdesign ist so einfach wie möglich – komplexes Design erzeugt schwer zu prüfende Lücken.
- **Fail-safe Defaults.** Im Zweifelsfall verweigert das System Zugriff, statt ihn zu erlauben.
- **Complete Mediation.** Jeder Zugriff wird geprüft, nicht nur der erste.
- **Separation of Privilege.** Sensible Operationen verlangen mehrere unabhängige Bedingungen.

### 6.2 Account- und Credential-Konzept

- **Eigene Bot-Accounts.** Bots dürfen nicht unter Mitarbeiter-Accounts laufen. Sonst ist im Audit unklar, wer was getan hat.
- **Vault-Pflicht.** Passwörter, API-Keys, Zertifikate werden in einem Credential Vault gespeichert. Bots holen Tokens, nie Klartext-Passwörter.
- **Least Privilege.** Wenn ein Bot nur lesen muss, bekommt er nur Leserechte. Nicht der “Standard-Account mit allem”.
- **Rotations-Disziplin.** Passwörter werden regelmäßig rotiert – automatisiert, nicht händisch.

### 6.3 Segregation of Duties (SoD)

Eine klassische Sicherheitsregel: kritische Aktionen werden nicht von einer Person (oder einem Bot) allein durchgeführt. “Anlegen” und “Freigeben” bleiben getrennt. Wer dieses Prinzip im manuellen Prozess hat, darf es im automatisierten nicht aufgeben – nur weil “der Bot ja zuverlässig ist”. (ISACA, 2018)

### 6.4 Logging-Integrität

Logs, die von dem Bot manipulierbar sind, dessen Aktionen sie protokollieren, sind kein Audit Trail. Drei Maßnahmen:

- **Append-only.** Logs werden geschrieben, nicht geändert.
- **Separater Account.** Der Bot-Account hat keine Schreibrechte auf den Log-Speicher; der Logging-Service hat sie.
- **Kryptographische Signatur.** Bei sensiblen Logs werden Einträge signiert; eine nachträgliche Änderung wäre erkennbar.

## 7. Change Management: vom Erfassen zum Entscheiden



Wer RPA einführt, ändert nicht “nur Tools”. Mitarbeitende, die früher manuell Rechnungen erfasst haben, müssen heute Exceptions bearbeiten und Bots überwachen. Das ist eine andere Tätigkeit – und sie wird nicht von selbst gut, wenn niemand sie unterstützt. (Kotter, 1996)

## 7.1 Der Rollenwandel

Vor RPA: “Ich erfasse Rechnungen.” Nach RPA: “Ich überwache Bots, bearbeite Ausnahmen und entscheide, wenn etwas nicht passt.”

Das ist eine *kognitiv anspruchsvollere* Tätigkeit. Sie verlangt:

- Verständnis des Gesamtprozesses (nicht nur des eigenen Schrittes).
- Entscheidungsfähigkeit unter Zeitdruck.
- Vertrauen in das eigene Urteil bei Ausnahmen.
- Bereitschaft, Verbesserungsvorschläge zu machen.

## 7.2 Kotter's 8 Steps – kompakt

John P. Kotter hat 1996 ein Modell für Change-Initiativen formuliert, das heute Standard ist. Übertragen auf RPA: (Kotter, 1996)

1. **Sense of Urgency.** Warum ist Veränderung jetzt nötig? Konkret, glaubwürdig.
2. **Guiding Coalition.** Wer treibt es? Eine kleine Gruppe mit Mandat und Energie.
3. **Vision.** Wie sieht der Endzustand aus? Ein Satz, der zieht.
4. **Communicate.** Häufig, in mehreren Kanälen, vor allem in den ersten Wochen.
5. **Empower.** Hindernisse beseitigen, Trainings anbieten, Entscheidungsbefugnis geben.
6. **Quick Wins.** Erste Erfolge sichtbar machen, früh und ehrlich.
7. **Sustain Acceleration.** Nicht nach den ersten Erfolgen aufhören – weiter ausrollen, weiter verbessern.
8. **Institutionalize.** Standards, Routinen, KPIs verankern, dass es nicht “wegbricht” wenn der Sponsor wechselt.

## 7.3 Akzeptanz: Transparenz, Nutzen, Würde

Drei Botschaften, die in jeder RPA-Einführung explizit kommuniziert werden müssen:

- **Transparenz.** Was tut der Bot, was nicht? Welche Daten sieht er? Wer prüft ihn?
- **Nutzen.** Was bringt der Bot *mir* als Mitarbeitendem – nicht nur dem Unternehmen?
- **Würde.** Der Bot nimmt repetitive Aufgaben, nicht “meinen Wert”. Das muss gezeigt werden, nicht nur gesagt.

**Eselsbrücke: T-N-W**

Drei Botschaften, die jede RPA-Einführung tragen müssen:

Transparenz – was tut der Bot, was nicht?

Nutzen – was bringt er *mir*?

Würde – Bots nehmen Aufgaben, nicht Wert.

*Ohne T-N-W bleibt jede Schulung Theaterdonner.*

## 8. Fallstudie: PharmaDistrib – RPA im regulierten Umfeld

*PharmaDistrib* ist ein pharmazeutischer Distributor. Das Unternehmen ist auditkritisch und steht unter strenger regulatorischer Aufsicht. RPA soll in Finance und Supply Chain ausgerollt werden: 10 Bots in 3 Monaten. Restriktionen: 10 Wochen bis zum Audit-Sampling, Budget 160 k EUR, begrenzte Security-Kapazität, Datenklassifikation teilweise unklar.

### 8.1 Minimum Viable Governance

Risk-based Releases:

- **Low-Risk-Bots** (keine sensiblen Daten, keine Buchungen): 1 Gate (QA grün) plus Bot-Owner-Approval.
- **Medium-Risk** (Daten ohne Personenbezug, kleine Buchungen): 2 Gates (QA + Process Owner).
- **High-Risk** (Pflichtige Buchungen, sensitive Daten): 3 Gates (QA + Security Owner + Process Owner).

Die RACI ist verbindlich; ein Bot ohne benannten Bot Owner darf nicht in Produktion.

### 8.2 Audit Trail Standard

Pflichtfelder pro Eintrag:

- Bot-ID + Version
- Eingang-Referenz (Mail-ID, Dokument-ID, Vorgangs-ID)
- Aktion (z. B. "Buchung Lieferanten-Rechnung X"),
- Ergebnisstatus (success, queued, exception),
- Exception-Code (falls relevant),
- Bearbeiter bei manueller Übernahme (User-ID),
- Reason (bei Ausnahme / Override),
- Ticket-ID (wenn aus Exception ein Ticket geworden ist).

Logs werden *zentral*, *append-only*, mit *Role-Based-Access* verwaltet. Aufbewahrung 5 Jahre (regulatorisch). Exportfähigkeit für Audit-Sampling ist vorgesehen.

### 8.3 Exception-Architektur

Codes:

- **Data.** MissingCostCenter, UnknownVendor, InvalidIBAN.
- **Business.** ApprovalRequired, DuplicateInvoice, ContractExpired.
- **System.** ERPDown, Timeout, ConnectionLost.
- **Security.** AuthFail, UnauthorizedField, VaultUnreachable.

Playbooks definieren Owner und SLA. Beispiel: “ERPDown → Retry 3× → Ticket an IT → Eskalation nach 2 h”. KPIs: Exception Rate  $\leq 12\%$  nach Stabilisierung; MTTR pro Exception  $< 4$  h.

## 8.4 Security-Guardrails

- Bot-Accounts getrennt, mit Least Privilege.
- Credentials im Vault, nie in Skripten oder Konfigurationen.
- SoD: Bot darf eine Anlage durchführen, aber keine Freigabe; Freigabe bleibt menschliche Entscheidung.
- Logging append-only, mit separater Storage-Identity.
- Quartalsweise Security-Review für High-Risk-Bots.

## 8.5 Change-Paket

Kommunikationsbotschaft: “Bots übernehmen Routine, Menschen übernehmen Entscheidungen und Exceptions.” Trainingspfad: zweistufig – Operator-Schulung (Exception-Bearbeitung, Playbook-Anwendung) und Owner-Schulung (Monitoring, KPI-Lesen, Verbesserung). Champions-Netzwerk: pro Bereich eine Person mit erweitertem Training, die im Alltag Fragen beantwortet. Feedbackkanal: monatlicher Pulse-Survey + offene Q&A.

## 8.6 Frühwarnsignale

- Exception Rate steigt über 18 % in einer Woche → Bot pausieren, Ursache analysieren.
- UI-Änderungen  $> 1$  pro Monat im selben Portal → Stabilität testen, ggf. API suchen.
- Audit-Findings im internen Sampling → sofortiges Review-Gate verschärfen.
- Anstieg “manuelle Übernahme”-Quote → Anzeichen, dass Vertrauen erodiert oder Datenqualität sinkt.

# 9. Coaching: Governance als Enablement

Der häufige Fehler reifer Programme: Governance wird zum Selbstzweck. Reviews dauern länger als der Bau, Standards werden detaillierter, jede Ausnahme braucht ein Komitee. Folge: die Fachbereiche *umgehen* die Governance – Schatten-Bots entstehen, das Vertrauen sinkt.

Senior-Disziplin: Governance als *Enablement* verstehen.

## 9.1 Drei Regeln einer leichten Governance

1. **Wenige Regeln, klar formuliert.** Lieber zehn Regeln, die jeder kennt, als hundert, die niemand findet.
2. **Risk-based.** Low-Risk schnell, High-Risk gründlich – nicht alles gleich behan-

deln.

3. **Messbar.** Jede Regel hat eine Messgröße. Was nicht gemessen wird, wird nicht eingehalten.

## 9.2 Audit Trail als Lernquelle

Ein gutes Audit-Trail ist nicht nur Compliance-Nachweis – es ist Debugging, Performance-Analyse und Verbesserungsquelle. Drei Anwendungen:

- **Debugging.** Bei einem Vorfall lässt sich Schritt für Schritt rekonstruieren, was passiert ist.
- **Performance.** Welche Schritte dauern am längsten? Wo sammeln sich Vorgänge in Queues?
- **Lernen.** Welche Exception-Klassen treten am häufigsten auf? Wo lohnt sich Investition in Robustheit?

## 9.3 Senior-Reflexionsfragen

- Welche 3 Guardrails verhindern “große Schäden” bei RPA am zuverlässigsten?
- Wo ist der Governance-Engpass (Security, QA, Fachfreigabe) – und wie wird er entlastet, ohne Standards aufzuweichen?
- Wie wird Verantwortlichkeit sichtbar, ohne Angstkultur zu erzeugen?

# 10. Management-Perspektive: CoE vs. Föderation

Eine zentrale strategische Frage in skalierender RPA: *CoE oder Föderation?*

## 10.1 CoE – zentrales Center of Excellence

Ein **Center of Excellence** ist ein dedizierter Bereich, der alle Bots zentral baut und betreibt. (Lacity & Willcocks, 2018)

- **Vorteile.** Klare Standards, einheitliche Qualität, gebündeltes Wissen, leichter Audit.
- **Nachteile.** Engpass, Distanz zur Fachseite, langsame Iteration, “Wir und die”-Dynamik.

## 10.2 Föderation – Bots in den Fachbereichen

Föderation bedeutet, dass Fachbereiche selbst Bots bauen (Citizen Developer) und betreiben, unter dezentralem Eigentum.

- **Vorteile.** Nähe zur Fachseite, schnelle Iteration, hohe Akzeptanz, lokale Verantwortung.
- **Nachteile.** Standardrisiko, Compliance-Risiko, Wartungs-Patchwork, Skalierbarkeitsprobleme.

## 10.3 Hybrides Modell – pragmatisch und häufig

In der Praxis ist die Antwort meist ein Hybridmodell: ein CoE setzt Standards, liefert Templates und QA, während dezentrale Teams bauen und betreiben. *Standards zentral, Bauen lokal, Audit zentral.*

Entscheidungs-Kriterien:

- Reifegrad der Organisation: Anfangs eher zentral, später föderiert.
- Regulatorischer Druck: Hoch reguliert tendiert zu CoE.
- Organisationskultur: Stark autonome Bereiche zu Föderation.
- Plattformreife: Reife Plattform mit Templates erlaubt mehr Föderation.

## 10.4 Risk Appetite definieren

Eine Senior-Entscheidung: *Wie viel Risiko ist akzeptabel?* Drei Dimensionen:

- **Compliance Risk.** Welche Findings sind tolerabel, welche nicht?
- **Operational Risk.** Wie häufig dürfen Bots ausfallen, bevor das Geschäft leidet?
- **Reputational Risk.** Welche Fehler dürfen *nie* öffentlich werden?

Diese Entscheidungen sind nicht delegierbar – sie gehören in das Steering-Komitee, ins Risk Office, in die Geschäftsführung.

### Schluss-Merksatz für Tag 12

RPA-Skalierung ist eine **Verantwortungsarchitektur**, keine Tool-Frage. Governance leicht, Audit Trail verbindlich, Security by Design, Change als Teil des Designs – vier Disziplinen, die zusammen ein wachsendes Programm tragen. Wer eine fortlässt, baut sich ein Schatten-Programm neben dem offiziellen.

## 11. Take-aways aus Tag 12

1. **Bot-Lifecycle in 9 Phasen:** Idea → Assessment → Design → Build → Test → Deploy → Run → Improve → Retire.
2. **Sechs Kernrollen:** Process Owner, Bot Owner, Developer, Reviewer/QA, Security, Operations.
3. **Risk-based Releases:** Low-Risk schnell (1 Gate), High-Risk gründlich (3 Gates).
4. **Audit Trail beantwortet die W-Fragen** und ist append-only, mit Aufbewahrung nach Regulatorik.
5. **Exception in drei Klassen:** Business, System, Data. Plus Security als vierte für sensible Bereiche.
6. **Security by Design:** Least Privilege, Vault, SoD, Logging-Integrität.
7. **Change Management nicht nachher:** Transparenz, Nutzen, Würde von Tag 1 an.
8. **CoE vs. Föderation:** Hybrid ist meist die Antwort – Standards zentral, Bauen lokal.

## 12. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

**International Organization for Standardization. (2022).**

*Information security, cybersecurity and privacy protection – Information security management systems – Requirements (ISO/IEC 27001:2022).* ISO.

**ISACA. (2018).**

*COBIT 2019 framework: Governance and management objectives.* ISACA.

**Kotter, J. P. (1996).**

*Leading change.* Harvard Business School Press.

**Lacity, M., & Willcocks, L. P. (2018).**

*Robotic process and cognitive automation: The next phase.* SB Publishing.

**Project Management Institute. (2021).**

*A guide to the Project Management Body of Knowledge (PMBOK® Guide) (7th ed.).* Project Management Institute.

**Saltzer, J. H., & Schroeder, M. D. (1975).**

The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>

**Smeets, M., Erhard, R., & Kaußler, T. (2019).**

*Robotic Process Automation (RPA) in der Finanzwirtschaft: Technologie – Implementierung – Erfolgsfaktoren.* Springer Gabler. <https://doi.org/10.1007/978-3-658-26564-2>

**Willcocks, L. P., Lacity, M., & Craig, A. (2017).**

Robotic process automation: Strategic transformation lever for global business services? *Journal of Information Technology Teaching Cases*, 7(1), 17–28. <https://doi.org/10.1057/s41266-016-0016-9>

## 13. Glossar

---

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

### **Append-only**

Log-Eigenschaft: Einträge können hinzugefügt, aber nicht geändert oder gelöscht werden. Schutz gegen Manipulation.

### **Audit Trail**

Lückenlose Aufzeichnung von Bot-Aktionen mit Bot-ID, Version, Aktion, Ergebnis, Begründung. Standard in regulierten Prozessen.

### **Bot Lifecycle**

Neunphasiger Zyklus: Idea, Assessment, Design, Build, Test, Deploy, Run, Improve, Retire.

### **Business Exception**

Fachlich begründete Ausnahme (z. B. ApprovalRequired, DuplicateInvoice). Erfordert fachliche Bearbeitung.

### **Center of Excellence (CoE)**

Zentraler Bereich, der RPA-Standards setzt und ggf. Bots zentral baut und betreibt.

### **Change Advisory**

Leichtgewichtiges Gremium, das teure oder sensible Bot-Änderungen reviewt.

### **Credential Vault**

Zentrales System für Zugangsdaten. Bots holen Tokens, nie Klartext-Passwörter.

### **Data Exception**

Datenqualitätsausnahme (z. B. MissingCostCenter, InvalidIBAN). Erfordert Korrektur an der Quelle oder im Bot.

### **Exception Playbook**

Handlungsanweisung für eine Exception-Klasse: Trigger, Diagnose, Aktionen, Eskalation, Schließung.

### **Fail-safe Defaults**

Sicherheits-Grundsatz: im Zweifel wird Zugriff verweigert. (Saltzer & Schroeder, 1975)

### **Fast Track**

Reduzierter Review-Pfad für Low-Risk-Bots. Reduktion der Reviews, nicht der Pflichtstandards.

### **Föderation**

Dezentrales RPA-Modell: Fachbereiche bauen und betreiben Bots in eigener Verantwortung.

### **Governance (RPA)**

Mechanismen zur Steuerung wachsender Bot-Portfolios: Vergleichbarkeit, Auffindbarkeit, Steuerbarkeit, Lernfähigkeit.

### **Least Privilege**

Sicherheitsprinzip: Ein Bot bekommt nur die Rechte, die er für seine Aufgabe braucht.



**Logging-Integrität**

Schutz von Logs vor nachträglicher Manipulation: append-only, separater Schreib-Account, ggf. Signatur.

**Minimum Viable Governance**

Kleinstmögliche Sammlung verbindlicher Regeln: Rollen, Gates, Standards, Monitoring, Incidents.

**Risk-based Releases**

Differenzierte Freigabewege je nach Bot-Risikoklasse (Low/Medium/High).

**Segregation of Duties (SoD)**

Trennung sensibler Tätigkeiten: “Anlegen” und “Freigeben” nicht durch eine Instanz.

**System Exception**

Technische Ausnahme (z. B. ERPDown, Timeout). Erfordert Retry oder IT-Eskalation.

**Traceability**

Durchgängige Nachverfolgbarkeit eines Vorgangs über alle Systeme, typischerweise über eine Korrelations-ID.

**W-Fragen-Katalog**

Audit-Standardfragen: Wer? Was? Wann? Womit? Ergebnis? Begründung?